

Examen primera evaluación:
Fecha : 01 de Diciembre 2025

**HECHO POR IZAN NAVARRO
LUJAN**

01/12/2025

Ejercicio 1: Testing Automatizado

Una empresa quiere mejorar la calidad de su aplicación web automatizando tests. Se te entrega el siguiente módulo JavaScript que contiene tres funciones:

```
// utils.js
function esMayorEdad(edad) {
  return edad >= 18;
}

function sumar(a, b) {
  return a + b;
}

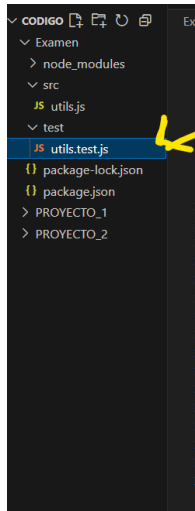
function obtenerDominio(url) {
  try {
    const u = new URL(url);
    return u.hostname;
  } catch {
    return null;
  }
}

module.exports = { esMayorEdad, sumar, obtenerDominio };
```

Resolver las siguientes tareas, utilizando Visual Studio Code.

1. Crea un archivo `test/utils.test.js`.

Genero el fichero dentro de la estructura de mi directorio:



2. Usando **Mocha y Chai**, implementa tests que verifiquen:
- Que `esMayorEdad(20)` devuelve `true`.
 - Que `esMayorEdad(15)` devuelve `false`.
 - Que `sumar(2, 3)` devuelve `5`.
 - Que `obtenerDominio("https://example.com/page")` devuelve `"example.com"`.

El código para la lógica que he desarrollado es este:

```
const Utils = require("../src/utils");
const { expect } = require("chai");

describe('Pruebas de los Utils', () => {

  it('Tener 20 es ser mayor de edad! (se espera true)', () => {
    expect(Utils.esMayorEdad(20) == true);
  });

  it('Tener 15 es ser menor de edad! (se espera false)', () => {
    expect(Utils.esMayorEdad(15) == false);
  });

  it('Sumar 2 + 3 = 5', () => {
    expect(Utils.sumar(2,3)).to.equal(5);
  });

  it('obtenerDominio debe devolver el dominio (si es "https://example.com/page" devuelve "example.com")', () => {
    expect(Utils.obtenerDominio("https://example.com/page"))
      .to.equal('example.com');
  });

});
```

Así comparamos todos los campos y valores

3. Ejecuta los tests y haz una captura de pantalla del resultado.

```
PS C:\Users\Izan Navarro\Desktop\CIBERSEGURIDAD\CE_Ciberseguridad\PRODUCCION_SEGURA\CODIGO\Examen2> npm test

> examen2@1.0.0 test
> mocha

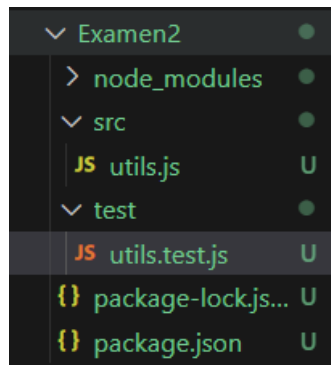
Pruebas de los Utils
✓ Tener 20 es ser mayor de edad! (se espera true)
✓ Tener 15 es ser menor de edad! (se espera false)
✓ Sumar 2 + 3 = 5
✓ obtenerDominio debe devolver el dominio (si es "https://example.com/page" devuelve "example.com")

4 passing (13ms)
```

Entregar :

1. Captura la pantalla con la estructura de directorios del proyecto cuando termines.

Abajo adjunto la estructuración de directorios del proyecto:



2. Fichero [utils.test.js](#)
(te lo adjunto en la entrega de este documento)
3. Captura de pantalla del resultado de las pruebas

```
PS C:\Users\Izan Navarro\Desktop\CIBERSEGURIDAD\CE_Ciberseguridad\PRODUCCION_SEGURA\CODIGO\Examen2> npm test

> examen2@1.0.0 test
> mocha

Pruebas de los Utils
✓ Tener 20 es ser mayor de edad! (se espera true)
✓ Tener 15 es ser menor de edad! (se espera false)
✓ Sumar 2 + 3 = 5
✓ obtenerDominio debe devolver el dominio (si es "https://example.com/page" devuelve "example.com")

4 passing (13ms)
```

Ejercicio 2: Determinación del nivel ASVS requerido por las Aplicaciones

La empresa "GreenTorrent" quiere lanzar una tienda online donde los usuarios podrán:

- Registrarse y autenticarse.
- Guardar datos de tarjeta (tokenizados por un proveedor externo).
- Comprar productos.
- Interactuar con un chatbot de soporte.

Se pide determinar **qué Nivel ASVS (1, 2 o 3)** es el adecuado para esta aplicación.

Tareas

1. Analiza el caso y justifica qué nivel ASVS es el más adecuado.
2. Especifica al menos **3 requisitos de ASVS** que la aplicación debería cumplir (pueden elegirse de cualquier categoría del Application Security Verification Standard 4.0.3, aunque debes intentar que sean relevantes para nuestra aplicación).
3. Plantea una pequeña matriz de test con 2 o 3 pruebas para sólo uno de los requisitos seleccionados en el punto número 2. Seguimos el OWASP Web Security Testing Guide (WSTG v4.2).
4. Explica por qué elegir un nivel inferior implicaría riesgos y pon un ejemplo de alguno de ellos.

Ejercicio 3: Revisión de vulnerabilidades en Juice Shop

Durante las pruebas de seguridad realizadas con **OWASP ZAP** sobre la aplicación "Juice Shop", se han identificado los siguientes comportamientos:

1. La página permite introducir `<script>alert(1)</script>` en el campo de comentarios.
2. La API de login devuelve información detallada sobre el error:

```
{ "error": "Usuario no encontrado en la base de datos" }
```
3. El endpoint `/rest/products/search?q=` permite hacer peticiones sin autenticación.

Tareas

Elige uno de los casos y responde a las siguientes preguntas:

1. Identifica el tipo de vulnerabilidad.
2. Explica por qué es un riesgo.
3. Indica **qué requisito de ASVS** se relaciona con ese problema.
4. Propón alguna medida de mitigación.